

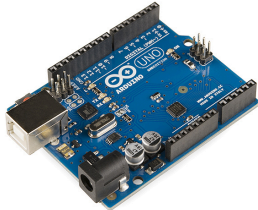
Project: AWBA (AC Water Bucket Alert)

Problem:

An air conditioner in a room produces condensation water that drips through a hose into a bucket placed on top of a cupboard. Because the bucket is not permanently connected to a drain, it needs to be manually emptied regularly. The problem is that the person in the room sometimes forgets to check and empty it, causing the bucket to overflow. This is made worse by the bucket's elevated position meaning when it gets too full, it becomes heavy and difficult to move safely without spilling.

Solution:

AWBA is an Arduino-based monitoring system that continuously tracks the water level in the bucket and alerts anyone in the room to empty it before it overflows when a set threshold is reached.

Components list	
Arduino UNO R3	
HC-SR04 Ultrasonic Sensor	

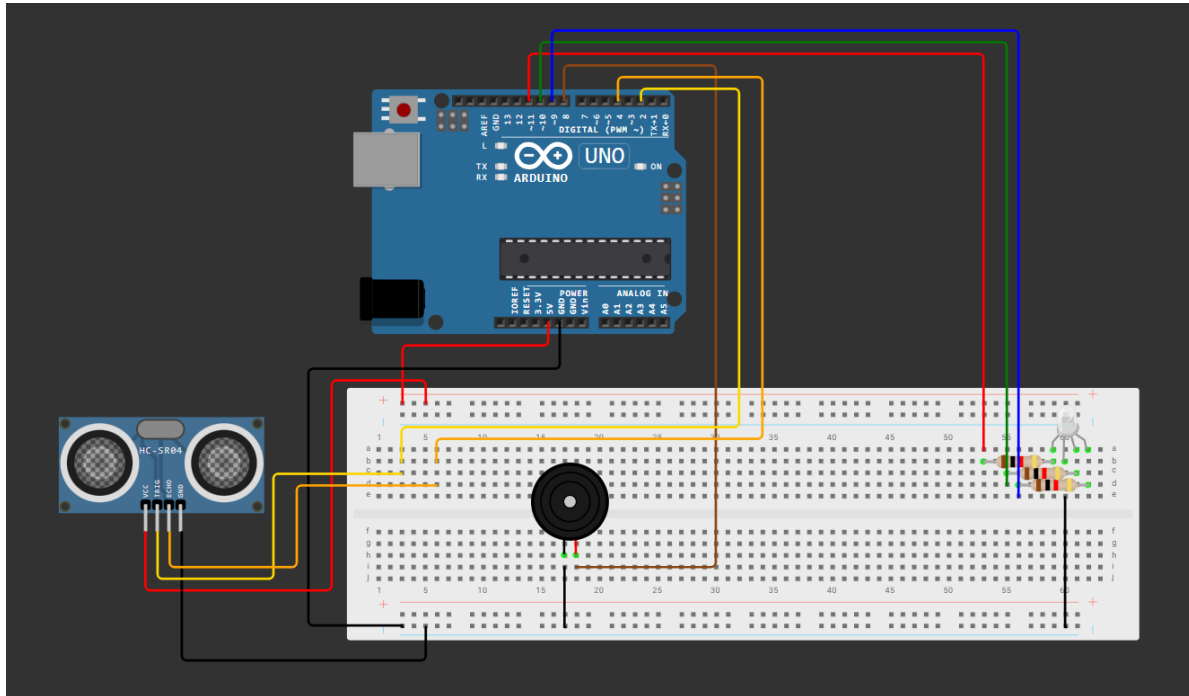
RGB LED Module (HW-479)	
Passive Buzzer	
9V Adapter	
220 ohm Resistors (x3)	

System design:

On startup the system waits one minute to allow time to position the sensor on the bucket. The HC-SR04 sensor then begins continuously measuring the distance between itself and the water surface. As the water level rises, the distance decreases. This distance is converted into a fill percentage based on the known empty bucket depth of 23cm. Below 50% the RGB shows green. From 50% to 74% it shifts to yellow as an early visual warning. At 75% or above a red flashing light and buzzer alarm activates for 15 seconds. At 90% or above the alarm becomes

more urgent with a faster strobe and higher pitched buzzer. After a 15-second alert, the system enforces a 5-minute cooldown where the buzzer remains silent, but the system still continuously monitors the water level.

Wiring Diagram:



1. HC-SR04

- VCC → Arduino 5V
- TRIG → Arduino D2
- ECHO → Arduino D4
- GND → Arduino GND

2. RGB Module (HW-479)

- R → 220 ohm resistor → Arduino D11
- G → 220 ohm resistor → Arduino D10
- B → 220 ohm resistor → Arduino D9
- (-) → Arduino GND

3. Passive Buzzer

- Positive (+) → Arduino D8
- Negative (-) → Arduino GND

Code:

```
#define TRIG 2
#define ECHO 4
#define BUZZER 8
#define RGB_R 11
#define RGB_G 10
#define RGB_B 9

// Empty bucket depth in cm, measured from sensor to bucket bottom
float emptyDistance = 23.0;
// Tracks the last time an alert was triggered, used to measure the 5 minute
cooldown
unsigned long lastAlertTime = 0;
// Cooldown to prevent repeated alerts
bool cooldown = false;

// Sets the RGB LED color using PWM values (0-255 per channel)
void setColor(int r, int g, int b) {
    analogWrite(RGB_R, r);
    analogWrite(RGB_G, g);
    analogWrite(RGB_B, b);
}

// Triggers the HC-SR04 and returns distance in cm
float getDistance() {
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);
    long duration = pulseIn(ECHO, HIGH);
    return duration * 0.034 / 2;
}

// Takes 5 readings and returns the average to reduce sensor noise
float getStableDistance() {
    float total = 0;
```

```

    for (int i = 0; i < 5; i++) {
        total += getDistance();
        delay(100);
    }
    return total / 5;
}

// Alerts for 15 seconds then starts a 5 minute cooldown
void triggerAlert(int buzzerFreq, int onTime, int offTime) {
    unsigned long start = millis();
    while (millis() - start < 15000) {
        tone(BUZZER, buzzerFreq);
        setColor(80, 0, 0);
        delay(onTime);
        noTone(BUZZER);
        setColor(0, 0, 0);
        delay(offTime);
    }
    noTone(BUZZER);
    setColor(0, 0, 0);

    // Start cooldown timer
    lastAlertTime = millis();
    cooldown = true;
}

void setup() {
    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);
    pinMode(BUZZER, OUTPUT);
    pinMode(RGB_R, OUTPUT);
    pinMode(RGB_G, OUTPUT);
    pinMode(RGB_B, OUTPUT);

    // Set RGB to solid blue to confirm the board has power and is booting
    setColor(0, 0, 255);

    // Wait 1 minute on startup to allow time to position the sensor on the

```

```

bucket
    delay(60000);

    // Clear the LED before handing control over to the main continuous loop
    setColor(0, 0, 0);
}

void loop() {
    // Check if 5 minute cooldown has passed
    if (cooldown && millis() - lastAlertTime >= 300000) {
        cooldown = false;
    }

    float distance = getStableDistance();

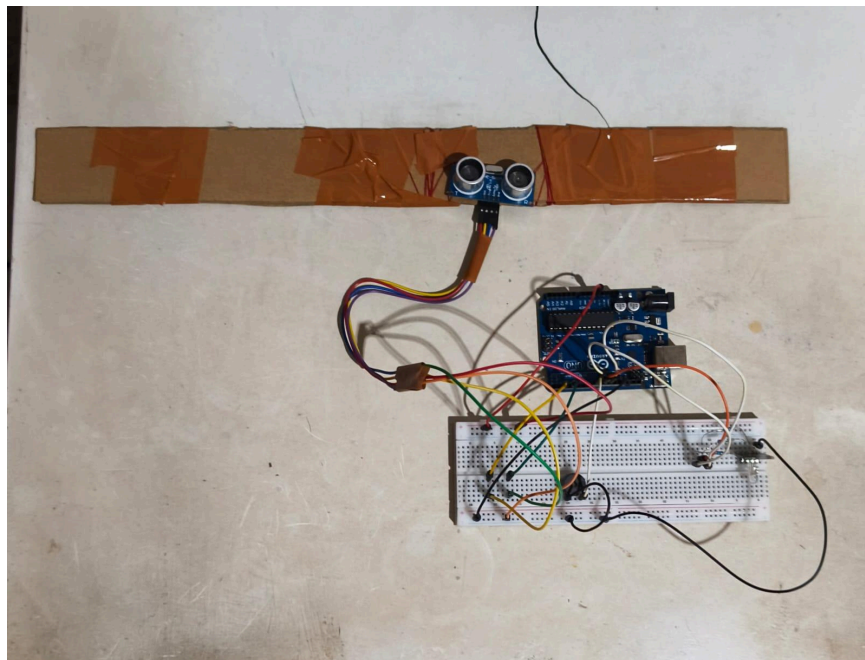
    // Calculate fill percentage based on how much distance has decreased
    float filled = (emptyDistance - distance) / emptyDistance * 100;
    filled = constrain(filled, 0, 100); // Clamp between 0 and 100

    if (!cooldown) {
        if (filled >= 90) {
            // Urgent alert: fast strobe and high pitch buzzer
            triggerAlert(2000, 150, 100);
        } else if (filled >= 75) {
            // Warning alert: slower strobe and lower pitch buzzer
            triggerAlert(1000, 300, 300);
        } else if (filled >= 50) {
            // Yellow: getting full, no alert yet
            setColor(230, 130, 0);
        } else {
            // Green: level is fine
            setColor(0, 255, 0);
        }
    }

    // Small delay between continuous checks
    delay(1000);
}

```

Documentation:





Limitations:

- The system requires a nearby power outlet due to the use of a 9V wall adapter
- The buzzer alert has no way to be acknowledged or silenced, it runs for the full 15 seconds regardless
- If the sensor is not aimed properly at the water surface it may give inaccurate readings

Future Improvements:

- Add a button to silence the alert once acknowledged
- Design a proper 3D printed bracket that mounts securely on the bucket rim and holds the sensor at a fixed angle pointing straight down at the center of the water surface, eliminating the risk of misaligned readings